UNIVERSIDAD CATÓLICA BOLIVIANA

INFORME

PRIMERA EVALUACIÓN

ASIGNATURA

Internet de las Cosas

INTEGRANTES

Álvaro Torrez

Karen Vargas

Docente	Ing. Pamela Valenzuela
Carrera	Ingeniería de Sistemas
Fecha de entrega	12 de octubre de 2026

LA PAZ – BOLIVIA

2026

Índice

1 Introducción 4
1.1 Descripción general del proyecto 4
1.2 Planteamiento del problema 4
1.3 Objetivos 4
1.3.1 Objetivo general 4
1.3.2 Objetivos específicos 4
1.4 Alcance 5
2 Diseño del Sistema IoT 6
2.1 Arquitectura general 6
2.2 Protocolos utilizados 7
2.3 Descripción de componentes y sensores 7
3 Diseño de la Base de Datos 9
3.1 Tipo de base de datos y justificación 9
3.2 Estructura de tablas y campos 9
3.2.1 `usuarios` 10
3.2.2 `preferencias_usuario` 10
3.2.3 `dispositivos` 10
3.2.4 `configuracion_dispositivo` 11
3.2.5 `salas` 11
3.2.6 `mapas` 11
3.2.7 `sesiones` 11
3.2.8 `lecturas` 11
3.3 Restricciones y tipos de datos 11
3.3.1 Validación de rangos (`CHECK`) 11
3.3.2 Índices únicos 12
3.3.3 Borrado en cascada (`ON DELETE`) 12
3.3.4 Automatismos del motor (**triggers** y vistas) 12
3.4 Seguridad (control de acceso a los datos) 12
4 Integración mediante API REST 14
4.1 Uso de la API REST 14
4.2 Inserción de datos (POST) 14
4.3 Lectura de datos (GET) 15
4.4 Manejo de errores 15
5 Desarrollo en MicroPython 16
5.1 Configuración de red 16
5.2 Lectura de datos (mínimo tres variables) 16
5.3 Envío de datos 16
5.4 Manejo de errores y reconexión 17
6 Sistema de Autenticación 18
6.1 Registro e inicio de sesión 18
6.2 Gestión de sesiones 18
6.3 Seguridad de acceso 18
7 Dashboard en Tiempo Real 19
7.1 Diseño del dashboard 19
7.2 Visualización de datos 19
7.3 Actualización en tiempo real 20

7.4 Interactividad 21
7.5 Manejo de errores 21
8 Conclusiones 22
8.1 Resultados obtenidos 22
8.2 Problemas y soluciones 22
8.3 Aprendizaje logrado 22
9 Anexos 24
9.1 Código fuente 24
9.2 Capturas de pantalla 24
9.3 Capturas del despliegue en producción 28
10 Referencias 34

1 Introducción

1.1 Descripción general del proyecto

El proyecto se trata de realizar una aspiradora automática con componentes asequibles, que no implique mucha inversión inicial. La solución terminó siendo un **robot aspirador doméstico de tipo Roomba**, con un **ESP32 programado en MicroPython** como cerebro físico. Por ahora el esfuerzo está puesto en resolver bien lo esencial —**moverse y evitar obstáculos**— y no todavía en la succión: el robot detecta paredes y muebles con tres sensores ultrasónicos, decide por sí mismo cómo esquivarlos, y manda esa telemetría en vivo a un dashboard web para que se pueda ver, desde el navegador, qué está haciendo en cada momento.

1.2 Planteamiento del problema

Los robots aspiradores comerciales con buena detección de obstáculos (sensores infrarrojos, LIDAR, cámaras) tienen un costo que los vuelve poco accesibles para construir o reparar de forma casera. El reto que plantea este proyecto es justamente el contrario: lograr una aspiradora automática con **detección de obstáculos confiable usando componentes lo más asequibles posible** —sensores ultrasónicos HC-SR04 en lugar de LIDAR comercial, un ESP32 en lugar de controladores propietarios— sin que esa reducción de costo signifique un robot que choque contra todo.

Con esa base de bajo costo funcionando, el problema se proyecta hacia una segunda etapa: la posibilidad de incorporar más adelante un **LIDAR casero**, construido con los mismos componentes económicos, que permita generar un mapeo del entorno más preciso que el que dan tres sensores ultrasónicos fijos, sin saltar directamente a un LIDAR comercial costoso.

1.3 Objetivos

1.3.1 Objetivo general

Prototipar una aspiradora automática con componentes asequibles, sin comprometer la detección de obstáculos.

1.3.2 Objetivos específicos

- Diseñar el esquema de conexiones de los componentes electrónicos en Fritzing, documentando el cableado real del robot con las piezas disponibles.
- Diseñar el esquema de base de datos (tablas, relaciones y reglas de integridad) que respalde la telemetría del robot y el historial de sus sesiones de limpieza.
- Implementar WebSockets para que la telemetría del robot se vea en el dashboard en tiempo real, sin depender de refrescar la página.
- Generar un dashboard con gráficas interactivas que permita visualizar correctamente los datos de telemetría a medida que llegan.
- Generar registros de las sesiones en las que el robot recorrió la habitación, para poder consultarlas después.
- Implementar inicio de sesión y autenticación de usuarios.
- Permitir que cada usuario registre y vincule su propio robot a su cuenta.
- Generar una versión beta de reconocimiento de la habitación mediante modelos de machine learning.

1.4 Alcance

En esta etapa se realiza un **prototipado real del robot**, no solo una simulación: el nodo ESP32 con sus sensores ultrasónicos y sus dos motores, construido y funcionando de forma física, publicando su telemetría por MQTT, junto con la base de datos PostgreSQL, el sistema de autenticación propio y el dashboard en tiempo real con su historial. A esto se suma una **beta de un sistema LIDAR casero**, como primer paso hacia un mapeo del entorno más fino que el que ofrecen los sensores ultrasónicos fijos.

También se logra la **visualización de datos en tiempo real** desde el dashboard, la **implementación de usuarios con dos roles** —administrador y usuario— y una página **desplegada en un VPS**, accesible desde cualquier dispositivo con conexión a internet.

Queda **fuera** del alcance de esta etapa —y así se deja documentado en el propio código, sin simular hardware que no existe—: el sistema de succión y los cepillos, sensores adicionales como el de polvo, la estimación de posición por odometría (el robot no tiene encoders), otros sensores adicionales (giroscopio, bumper, caída) y cualquier estrategia de cobertura de limpieza.

2 Diseño del Sistema IoT

2.1 Arquitectura general

El sistema sigue una arquitectura de **tres capas**: nodo IoT, backend y dashboard. Cada lectura del robot se retransmite por WebSocket al instante y solo después se encola para guardarse en PostgreSQL en lotes de un segundo.

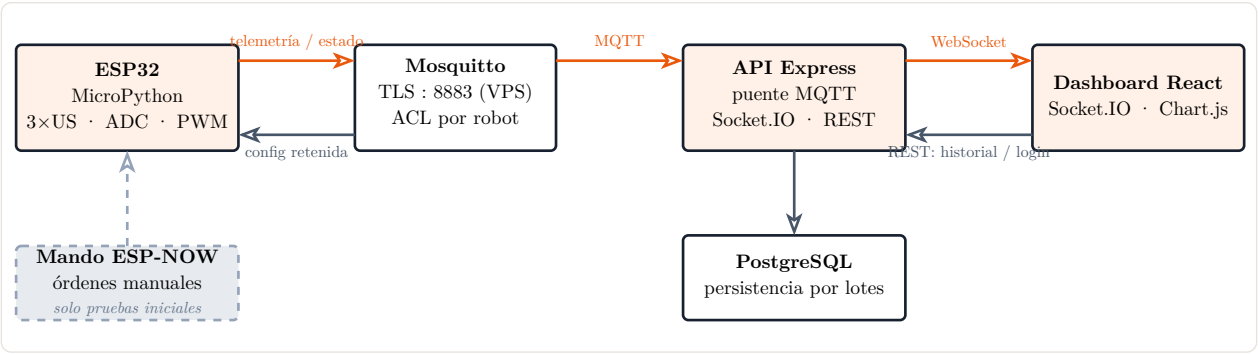


Figura 1: Arquitectura general del sistema: nodo IoT, transporte, backend y dashboard.

El flujo tiene cuatro tramos:

1. **ESP32 → Mosquitto**: telemetría por MQTT (puerto 8883), con conexión persistente y **Last Will** para anunciar desconexión.
2. **Mosquitto → API**: la API está suscrita a Mosquitto y recibe ahí cada mensaje que publica el robot; lo valida y lo guarda en orden, uno por uno, para que no se mezclen las lecturas de dos robots distintos.
3. **API → Dashboard**: cada lectura válida se emite de inmediato por Socket.IO a la sala del robot (`robot:{id}`) y **solo después** entra en el buffer que se vacía a PostgreSQL una vez por segundo.
4. **API → PostgreSQL**: persistencia por lotes, pensada para no hacer un `INSERT` por mensaje cuando el robot publica varias veces por segundo.

2.2 Protocolos utilizados

Protocolo	Tramo	Para qué
MQTT (puerto 8883)	ESP32 ↔ Mosquitto ↔ API	Lleva la telemetría y el estado de conexión. Si el robot se cae, el broker avisa solo, sin esperar a que nadie pregunte
WebSocket (Socket.IO)	API ↔ Dashboard	Envía los datos en vivo al dashboard apenas llegan, ya autenticado desde que se abre la conexión
HTTP/REST (JSON)	Dashboard ↔ API	Lo que no es tiempo real: iniciar sesión, ver el historial y descargar el CSV
ESP-NOW *(solo pruebas)*	Mando ↔ ESP32	Órdenes manuales de reacción rápida, sin pasar por el Wi-Fi ni por el backend; un prototipo inicial, no parte del flujo final

Tabla 1: Protocolos de transporte usados en cada tramo del sistema.

La razón de usar MQTT en vez de solo REST para la telemetría se apoya en cuatro motivos: **latencia** (una conexión persistente evita abrir una conexión nueva en cada envío a 2 Hz), **configuración sin sondeo** (el robot se suscribe a `config` y recibe el cambio sin tener que preguntar), **detección de caída gratuita** (el **Last Will** publica `en_linea: false` sin esperar un timeout) y **configuración retenida** (un robot que arranca, o un dashboard que se abre, ven el estado vigente sin pedirlo).

2.3 Descripción de componentes y sensores

El nodo físico es un **ESP32** con los siguientes componentes (pines reales de `firmware/configuracion.py`):

Componente	Función
ESP32	Microcontrolador principal: corre MicroPython y coordina sensores, motores y comunicación
3× HC-SR04 (ultrasónico)	Distancia a obstáculos en los tres frentes del robot (izquierda, centro, derecha)
TB6612FNG (puente H)	Controla dirección y velocidad de los dos motorreductores por PWM, con sentido independiente por rueda
Divisor resistivo	Adapta los 5 V de las líneas ECHO y de la batería a los 3.3 V que tolera el ESP32
2× pilas 18650	Fuente de energía del robot (6.0–8.4 V)
Portapilas	Alojamiento y conexión de las dos pilas 18650
LM2596	Regulador que reduce el voltaje de la batería al riel de 5 V que usan el ESP32, el driver y los sensores
2× motorreductores con ruedas	Tracción diferencial del robot

Tabla 2: Componentes y sensores del nodo IoT.

Espacio reservado para el diagrama de Fritzing (pendiente de insertar)

Figura 2: Diagrama de conexión: ESP32, driver de motores, sensores ultrasónicos y batería.

3 Diseño de la Base de Datos

3.1 Tipo de base de datos y justificación

El sistema usa **PostgreSQL** nativo en el VPS (sin Docker), administrado directamente por la API con el cliente `pg`. Se eligió PostgreSQL —y no una solución gestionada tipo Supabase— porque el proyecto corre en un VPS pequeño ya ocupado por Mosquitto, Node.js y Nginx/Caddy, y porque las reglas de integridad del dominio (rangos de PWM y distancia, una sola sesión activa por robot, configuración obligatoria por robot) se expresan de forma natural con restricciones `CHECK`, índices únicos parciales y **triggers** del propio motor relacional, sin depender de una capa externa.

3.2 Estructura de tablas y campos

Tabla	Qué guarda
`usuarios`	Cuentas que entran al dashboard: correo, hash bcrypt de la contraseña, nombre, rol (`admin` / `cliente`), si está activa y si tiene el permiso del modo lidar
`preferencias_usuario`	1 a 1 con `usuarios`: tema e interfaz del panel (qué widgets, dónde y con qué ajustes) como JSONB opaco, para que la misma cuenta vea lo mismo en cualquier dispositivo
`dispositivos`	Un robot físico. Su `id` de texto es a la vez el usuario MQTT (`roomba-sala`, …), con hash y cifrado del token, estado de conexión y versión de firmware
`configuracion_dispositivo`	1 a 1 con `dispositivos`: PWM de crucero, ángulos y geometría de los sensores, calibración de movimiento y qué habitación está mapeando ahora. Es lo único que el robot «obedece»
`salas`	Una habitación de un robot en modo lidar: su polígono de paredes, el tamaño estimado por el usuario y el real una vez mapeada, y las puertas (huecos que no son salida)
`mapas`	1 a 1 con `salas`: la rejilla de ocupación acumulada por el barrido láser, sus obstáculos ya extraídos y cuánta superficie está cubierta
`sesiones`	Un ciclo de funcionamiento del robot: inicio, fin (`NULL` = activa), total de lecturas y batería al inicio/fin
`lecturas`	Telemetría: una fila por mensaje publicado por el ESP32 — distancia de cada sensor, PWM con signo de cada rueda, voltaje y porcentaje de batería

Tabla 3: Tablas del esquema relacional.

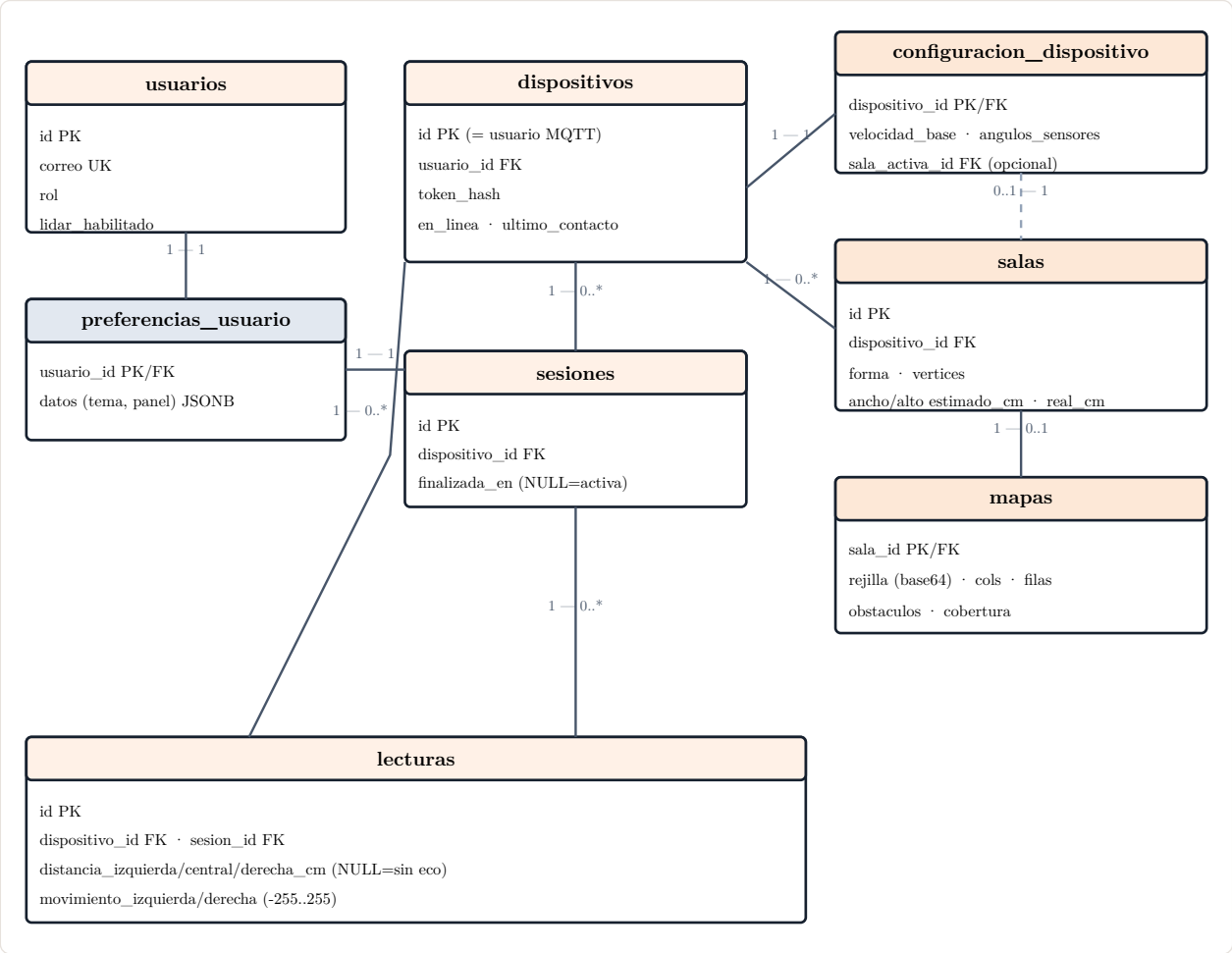


Figura 3: Diagrama entidad-relación del esquema (`apps/api/db/schema.sql`).

3.2.1 `usuarios`

Guarda la identidad de quien entra al dashboard: correo, hash bcrypt de la contraseña, nombre, rol (`admin` ve todos los robots, `cliente` solo los suyos), si la cuenta está activa y si tiene el permiso extra del modo lidar. Fila de ejemplo:

```
{id: 7, correo: "ana@demo.com", contrasena: "$2b$10$…", rol: "cliente",
 activo: true, lidar_habilitado: true}
```

3.2.2 `preferencias_usuario`

Relación 1 a 1 con `usuarios`. Es la única tabla pensada para que la lea y escriba el navegador casi sin validar: tema e interfaz del panel (qué widgets, dónde y con qué ajustes) como JSONB opaco, para que la misma cuenta vea lo mismo en cualquier dispositivo. Fila de ejemplo:

```
{usuario_id: 7, datos: {"tema": "oscuro", "panel": {"visibles": ["radar", "bateria"]}}}
```

3.2.3 `dispositivos`

El robot físico en sí. Su `id` de texto es a la vez el usuario MQTT (`roomba-sala`, …), con hash y cifrado del token, estado de conexión y versión de firmware. Fila de ejemplo:

```
{id: "roomba-sala", usuario_id: 7, nombre: "Roomba Sala",
 token_hash: "$2b$10$…", en_linea: true}
```

3.2.4 `configuracion_dispositivo`

Relación 1 a 1 con `dispositivos` y la única tabla que el firmware **obedece**: todo lo demás es lo que el robot **reporta**. Guarda el PWM de crucero, ángulos y geometría de los sensores, calibración de movimiento y qué habitación está mapeando ahora (`sala_activa_id`, único enlace del robot hacia el modo lidar). Fila de ejemplo:

```
{dispositivo_id: "roomba-sala", velocidad_base: 180,
 angulos_sensores: {"izquierdo": -45, "central": 0, "derecho": 45}, sala_activa_id: 3}
```

3.2.5 `salas`

Una habitación de un robot en modo lidar: su polígono de paredes, el tamaño estimado por el usuario y el real una vez mapeada, y las puertas (huecos que no son salida). Fila de ejemplo:

```
{id: 3, dispositivo_id: "roomba-sala", nombre: "Sala", forma: "rectangulo",
 ancho_estimado_cm: 400, alto_estimado_cm: 350}
```

3.2.6 `mapas`

Relación 1 a 1 con `salas`: la rejilla de ocupación acumulada por el barrido láser, sus obstáculos ya extraídos y cuánta superficie está cubierta. Se guarda aparte de `salas` porque pesa mucho más que el resto de la fila. Fila de ejemplo:

```
{sala_id: 3, cols: 80, filas: 70, resolucion_cm: 5, barridos: 214, cobertura: 0.86}
```

3.2.7 `sesiones`

Un ciclo de funcionamiento del robot: inicio, fin (`NULL` = activa), total de lecturas y batería al inicio/fin. Agrupa la telemetría de un mismo ciclo de limpieza, para poder preguntar «¿cómo le fue a esta sesión?» sin agregar manualmente miles de `lecturas`. Fila de ejemplo:

```
{id: 152, dispositivo_id: "roomba-sala", iniciada_en: "2026-10-10T08:00Z",
 finalizada_en: null, bateria_inicio_porcentaje: 96}
```

3.2.8 `lecturas`

Telemetría cruda: una fila por mensaje publicado por el ESP32 — distancia de cada sensor, PWM con signo de cada rueda, voltaje y porcentaje de batería. Es la tabla que más crece, la materia prima de todas las gráficas del panel. Fila de ejemplo:

```
{id: 48213, dispositivo_id: "roomba-sala", sesion_id: 152, distancia_central_cm: 18.0,
 movimiento_izquierda: 180, movimiento_derecha: 180, bateria_porcentaje: 94}
```

3.3 Restricciones y tipos de datos

Las reglas de integridad viven en la propia base, no solo en el código de la API. Se agrupan en cuatro familias:

3.3.1 Validación de rangos (`CHECK`)

Impiden que se guarde un valor fuera de lo físicamente posible, directamente en la base.

Regla	Qué garantiza
`rol IN ('admin','cliente')` en `usuarios` `forma IN ('rectangulo','poligono')` en `salas`	Solo esos valores son válidos para el campo; cualquier otro lo rechaza la base, no la API.
`movimiento_izquierda/movimiento_derecha BETWEEN -255 AND 255` en `lecturas`	El PWM con signo nunca puede salirse de la escala real del driver de motores.
`distancia_*_cm BETWEEN 0 AND 400` en `lecturas`	Rango físico del sensor HC-SR04. `NULL` significa explícitamente «el eco no volvió», no «falta el dato».
`bateria_porcentaje BETWEEN 0 AND 100`	Un porcentaje de batería fuera de ese rango no tiene sentido.
`ancho_estimado_cm/alto_estimado_cm BETWEEN 50 AND 2000` en `salas`	Una habitación fuera de ese rango es casi con certeza un error de tecleo, no una sala real.

Tabla 4: Restricciones `CHECK` por tabla.

3.3.2 Índices únicos

Evitan duplicados o estados inconsistentes que ninguna validación de la API podría impedir con la misma seguridad.

Índice	Qué garantiza
`uniq_sesion_activa_por_dispositivo` (parcial, `WHERE finalizada_en IS NULL`)	Un mismo robot no puede tener dos sesiones abiertas a la vez.
`uniq_sala_nombre_por_dispositivo` (`dispositivo_id, lower(nombre)`)	Dos habitaciones del mismo robot no pueden llamarse igual, sin distinguir mayúsculas.

Tabla 5: Índices únicos y qué duplicado evitan.

3.3.3 Borrado en cascada (`ON DELETE`)

`ON DELETE CASCADE` recorre toda la jerarquía usuarios → dispositivos → configuración / salas / sesiones → mapas / lecturas: borrar una cuenta no deja filas huérfanas en ninguna tabla dependiente. La única excepción es `configuracion_dispositivo.sala_activa_id`, con `ON DELETE SET NULL`: borrar la sala que el robot tenía activa no debe borrar ni bloquear el robot, solo dejarlo sin habitación asignada.

3.3.4 Automatismos del motor (triggers y vistas)

- **Trigger** `trg_configuracion_por_defecto`: al insertar un `dispositivo`, crea automáticamente su fila de `configuracion_dispositivo`, así el resto del código puede asumir que siempre existe.
- Tres **vistas** (`v_lecturas_por_minuto`, `v_lecturas_por_hora`, `v_resumen_sesion`) agregan la telemetría en el propio motor de base de datos, para no mandar miles de filas crudas al navegador en cada gráfica.

3.4 Seguridad (control de acceso a los datos)

El esquema no usa **Row Level Security** de PostgreSQL porque el acceso se decide en la API, no en consultas directas del cliente a la base: el dashboard nunca habla con PostgreSQL directamente, solo con la API REST y el WebSocket, que son los únicos procesos con credenciales de base de datos. La política de acceso equivalente a RLS se aplica en dos capas:

1. **Autenticación**: todo endpoint de datos exige un JWT válido (`requireAuth`), que se revalida contra la tabla `usuarios` en cada petición —no solo contra la firma del token— para que una cuenta desactivada o con el rol cambiado pierda el acceso de inmediato, sin esperar a que caduque el token.
2. **Autorización por propiedad**: `exigirPropiedad(dispositivoId, usuario)` comprueba en cada ruta de `datos.ts` que el robot pedido pertenece al usuario autenticado, salvo que su rol sea `admin`. La misma comprobación (`puedeVer`) se aplica al unirse a una sala de Socket.IO, para que la telemetría de un robot no llegue a un navegador que no es su propietario.

Las contraseñas de usuario se guardan con **bcrypt** (factor de coste 10) y nunca en claro; el token MQTT de cada robot también se guarda como hash bcrypt, y solo se muestra en texto claro una vez, al crear el robot desde el dashboard.

4 Integración mediante API REST

4.1 Uso de la API REST

La API expone rutas bajo el prefijo `/api`, agrupadas por responsabilidad: `auth.ts` (registro/login), `dispositivos.ts` (alta y gestión de robots), `datos.ts` (lecturas, sesiones, configuración, estadísticas) e `ingesta.ts` (inserción alternativa por HTTP). Todas las rutas de datos pasan por el middleware `requireAuth` y, cuando aplica, por `exigirPropiedad`.

Método y ruta	Función
`POST /api/auth/signup` · `/auth/login`	Registro e inicio de sesión; devuelven un JWT
`POST /api/ingesta`	Inserción de lotes de lecturas vía REST, autenticada con el token del robot
`GET /api/dispositivos/:id/lecturas`	Lectura cruda o agregada (`raw` / `minuto` / `hora`) por rango de fechas
`GET /api/dispositivos/:id/lecturas/export.csv`	Exportación a CSV del mismo rango
`GET /api/dispositivos/:id/sesiones`	Historial de sesiones de limpieza del robot
`GET` / `PATCH /api/dispositivos/:id/configuracion`	Lectura y cambio del PWM de crucero y los ángulos de los sensores
`GET /api/dispositivos/:id/resumen`	Estadísticas del día y acumuladas: tiempo en marcha, batería actual, mensajes por segundo
`GET /api/salud`	Estado de PostgreSQL, Mosquitto, tamaño del buffer de persistencia y clientes WebSocket conectados

Tabla 6: Rutas principales de la API REST (`apps/api/src/routes/`).

4.2 Inserción de datos (POST)

`POST /api/ingesta` acepta lotes de hasta 200 lecturas, validadas con el mismo esquema Zod que usa el puente MQTT (`telemetriaSchema`), para que ambos caminos de entrada —MQTT y REST— terminen en la misma función `procesarTelemetria`:

```
POST /api/ingesta
Authorization: Bearer SalaDemoToken0123456789abcdefgh
Content-Type: application/json

{
  "dispositivoId": "roomba-sala",
  "lecturas": [{
    "distancias_cm": { "izquierdo": 42.5, "central": 18.0, "derecho": null },
    "bateria_v": 7.8,
    "motores": { "izquierda_pwm": 180, "derecha_pwm": 180 }
  }]
}
```

El token se compara con `bcrypt.compare` contra el hash guardado en `dispositivos.token_hash`, con una caché en memoria de 5 minutos por robot para no pagar el coste de bcrypt (100 ms) en cada lote recibido cada segundo.

4.3 Lectura de datos (GET)

```
curl -H "Authorization: Bearer <jwt-de-usuario>" \
  "https://<host>/api/dispositivos/roomba-sala/lecturas?
agregacion=minuto&desde=2026-10-10T00:00:00Z"
```

El parámetro `agregacion` decide si la respuesta sale de la tabla cruda `lecturas` (para ventanas cortas, p. ej. el panel en vivo) o de una de las vistas agregadas `v_lecturas_por_minuto` / `v_lecturas_por_hora` (para rangos largos, como el historial), con un límite máximo de 5000 filas por respuesta.

4.4 Manejo de errores

Todas las rutas comparten un error handler centralizado (`http.ts`): los errores de validación Zod se traducen a `400` con un mensaje por campo, los errores de negocio se lanzan como `HttpError(status, mensaje)` con el código HTTP correcto (`401` sin token o credenciales incorrectas, `403` cuenta desactivada o ruta de administrador, `404` recurso inexistente, `409` correo duplicado) y cualquier error no previsto cae en un `500` genérico que se registra en el log del servidor sin filtrar detalles internos al cliente.

```
export const errorHandler: ErrorRequestHandler = (err, _req, res, _next) => {
  if (err instanceof HttpError) {
    res.status(err.status).json({ error: err.message, fields: err.fields });
    return;
  }
  // ...
  res.status(500).json({ error: "Error interno del servidor." });
};
```

El puente MQTT aplica la misma filosofía del lado del transporte: ningún mensaje mal formado puede tumbar el proceso. Un payload que no es JSON válido, o que no pasa el esquema Zod, se registra en el log y se descarta; el robot que lo envió sigue publicando normalmente en el siguiente ciclo.

5 Desarrollo en MicroPython

5.1 Configuración de red

`firmware/conexion_wifi.py` conecta el ESP32 en modo estación (`network.STA_IF`) con un **timeout** explícito de 10 segundos; si no logra conectar, lanza `OSError` y el programa principal lo captura sin detenerse: el robot sigue esquivando obstáculos y aceptando órdenes por ESP-NOW aunque no tenga Wi-Fi ni MQTT.

```
def conectar_wifi():
    wifi = network.WLAN(network.STA_IF)
    wifi.active(True)
    if not wifi.isconnected():
        wifi.connect(cfg.WIFI_SSID, cfg.WIFI_CLAVE)
        inicio = time.ticks_ms()
        while not wifi.isconnected():
            if time.ticks_diff(time.ticks_ms(), inicio) > 10000:
                raise OSError("Tiempo agotado al conectar Wi-Fi")
            time.sleep_ms(100)
    return wifi
```

5.2 Lectura de datos (mínimo tres variables)

El bucle principal (`main.py`) actualiza **un sensor ultrasónico por iteración** (ronda rotativa entre izquierdo, central y derecho cada ≥15 ms) para no bloquear el resto del programa con los hasta 12 ms que puede tardar un `HC-SR04` en responder, y lee la batería en la misma cadencia:

```
if time.ticks_diff(ahora, ultima_lectura) >= 15:
    ultima_lectura = ahora
    sensores.actualizar_un_sensor()
    voltaje_bateria, porcentaje_bateria = sensores.leer_bateria()
```

Las **tres variables mínimas exigidas** que el robot reporta son, por tanto: distancia de cada uno de los tres sensores (`distancias_cm`), voltaje/porcentaje de batería (`bateria_v`, `bateria_porcentaje`) y el PWM con signo de cada motor (`motores.izquierda_pwm/derecha_pwm`), además del estado del receptor ESP-NOW.

5.3 Envío de datos

Cada `INTERVALO_MQTT_MS` (500 ms por defecto, 2 Hz) el firmware publica un único JSON por MQTT en `roomba/{ROBOT_ID}/telemetria`:

```
datos = {
    "robot_id": cfg.ROBOT_ID,
    "distancias_cm": sensores.obtener_distancias(),
    "bateria_v": voltaje_bateria,
    "bateria_porcentaje": porcentaje_bateria,
    "motores": motores.obtener_estado(),
    "espnow": radio.obtener_estado(),
}
mqtt.publicar_telemetria(datos)
```

Las claves del diccionario son las mismas que valida el backend y las mismas que terminan como columnas en PostgreSQL: el esquema de la base **es** el JSON del robot, sin traducir nombres por el camino.

5.4 Manejo de errores y reconexión

`ClienteMQTT` (`comunicacion_mqtt.py`) encapsula tres mecanismos de resiliencia:

1. **Last Will**: al conectar, registra un mensaje retenido (`en_linea: false`) que Mosquitto publica automáticamente si el robot se desconecta sin avisar (corte de energía, pérdida de señal). El backend lo usa para marcar el robot fuera de línea sin necesitar un **timeout**.
2. **Reconexión con espera fija de 5 s**: `intentar_reconectar()` se llama en cada vuelta del bucle principal; si no está conectado y pasaron ≥5000 ms desde el último intento, reconecta. No bloquea el bucle: los sensores y motores siguen funcionando mientras tanto.
3. **Degradación explícita**: si `umqtt.simple` no está instalado, o Wi-Fi no conecta, el error se captura en `main.py` y se imprime, pero el programa continúa. La prioridad en el bucle es siempre: 1) órdenes ESP-NOW y seguridad, 2) sensores, 3) telemetría MQTT — en ese orden, así que un fallo de red nunca compromete la evasión de obstáculos.

```
def intentar_reconectar(self):
    if self.conectado:
        return
    if time.ticks_diff(time.ticks_ms(), self.ultimo_intento_ms) < 5000:
        return
    self.ultimo_intento_ms = time.ticks_ms()
    try:
        self.conectar()
    except Exception as error:
        self.desconectar()
        print("MQTT no disponible:", error)
```

Adicionalmente, `motores.py` aplica un **tiempo de parada de seguridad** (`TIEMPO_PARADA_MS = 250`): si no llega ninguna orden nueva en ese plazo, el robot detiene los motores, para no seguir avanzando a ciegas si pierde la comunicación con su mando.

6 Sistema de Autenticación

6.1 Registro e inicio de sesión

El registro (`POST /api/auth/signup`) valida con Zod un nombre (2–80 caracteres), un correo (normalizado a minúsculas) y una contraseña de al menos 8 caracteres con letra y número, y confirma que coincide con su repetición. La contraseña se guarda con **bcrypt** (factor 10); toda cuenta nueva se crea con rol `cliente`. El inicio de sesión (`POST /api/auth/login`) compara la contraseña con `bcrypt.compare` y rechaza cuentas desactivadas (`activo = false`) con un `403` explícito.

```
const hash = await bcrypt.hash(body.password, 10);
await pool.query(
  `INSERT INTO usuarios (correo, contrasena, nombre, rol)
   VALUES ($1, $2, $3, 'cliente') RETURNING ...`,
  [body.email, hash, body.name],
);
```

6.2 Gestión de sesiones

La sesión de usuario se mantiene con un **JWT firmado** (`jsonwebtoken`, `HS256`), con expiración de 12 horas, emitido en login/registro y enviado por el cliente en la cabecera `Authorization: Bearer <token>`. A diferencia de un JWT «ciego», `userFromToken` **relee el usuario en PostgreSQL en cada petición** (`WHERE id = $1 AND activo`): si un administrador desactiva una cuenta o le cambia el rol, el acceso se corta en la siguiente petición, sin esperar a que el token expire. La misma verificación protege el **handshake** de Socket.IO, y `disconnectUser(id)` desconecta activamente los sockets abiertos de un usuario que deja de tener acceso.

6.3 Seguridad de acceso

- Contraseñas y tokens de robot nunca se guardan en claro (bcrypt).
- El rol (`admin` / `cliente`) se valida tanto a nivel de esquema (`CHECK`) como de aplicación (`requireAdmin` como middleware de Express).
- Separación de identidades: el JWT de usuario autentica al **dashboard**; el token bcrypt del robot autentica al **ESP32** (vía MQTT, con su propia credencial en Mosquitto, o vía `Bearer` en `POST /api/ingesta`). Un ESP32 nunca inicia sesión como usuario.
- Autorización por propiedad en cada recurso (`exigirPropiedad`), no solo autenticación: estar logueado no implica poder ver los robots de otra cuenta.
- Un usuario `admin` puede resetear la contraseña de otro usuario desde el panel de administración: cubre la recuperación de cuenta mientras el flujo de correo (`/recuperar`) no tiene un servidor SMTP configurado, limitación que el propio `README.md` declara abiertamente en vez de simular un envío de correo que no ocurre.

7 Dashboard en Tiempo Real

7.1 Diseño del dashboard

El frontend (`apps/web`) es una aplicación React + Vite + TailwindCSS, estructurada en páginas (`Login`, `Registro`, `Panel`, `Historial`, `Sesiones`, `Robots`, `Perfil`, y el panel de administración `PanelAdmin` / `Usuarios` / `RobotsAdmin`) y un conjunto de componentes de tarjeta reutilizables para cada magnitud: `TarjetaBateria`, `TarjetaDistancias`, `TarjetaBarrasDistancia`, `RadarSensores`, `TarjetaActividadMotor`, `TarjetaConectividad`, `TarjetaSesiones`, `TarjetaContador` y `TarjetaEstado`.

7.2 Visualización de datos

Las gráficas se construyen con **Chart.js**, registrado una sola vez en `lib/chart.ts` para evitar el error clásico de «controller no registrado» al añadir un gráfico nuevo. El panel combina varios tipos de visualización sobre la misma telemetría:

- **Radar** de los tres sensores de distancia, posicionado según los ángulos reales de montaje (`angulos_sensores`), coloreado con los umbrales `DIST_EVASION_CM = 15 cm` (rojo) y `DIST_PRECAUCION_CM = 30 cm` (ámbar).
- **Serie de líneas** de distancia en el tiempo, con zoom y **pan** (`chartjs-plugin-zoom`).
- **Gauge** de batería.
- **Barra de cadencia** de conectividad (última lectura frente a la cadencia esperada).

Las siguientes son capturas reales del panel en producción (`https://iot-dashboard.tumype.com/`, cuenta `alvaro@demo.com`). Como en ese momento no hay ningún robot ni simulador publicando, las gráficas aparecen vacías — es el estado real que vería cualquier visitante del enlace, no uno maquillado con datos de relleno:

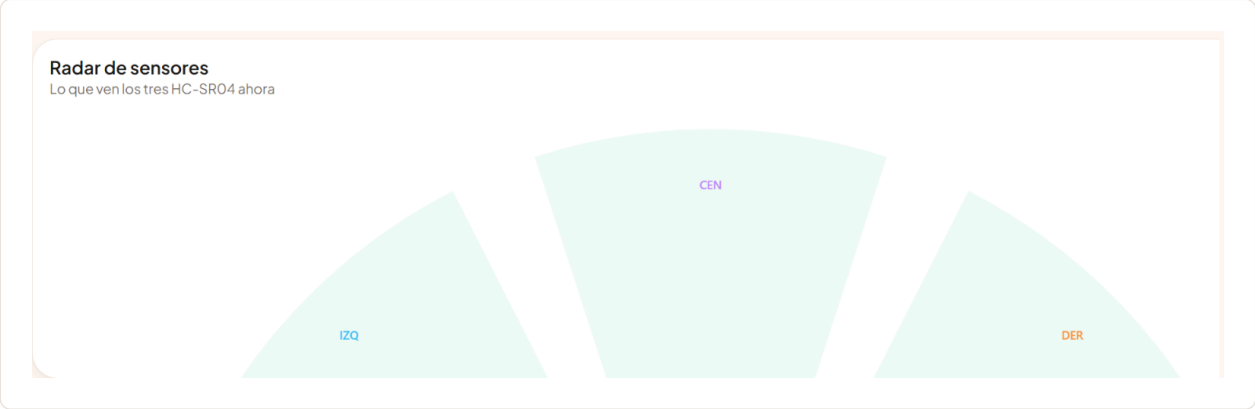


Figura 4: Radar de sensores, con los tres sectores (izquierdo, central, derecho) según los ángulos reales de `angulos_sensores`.

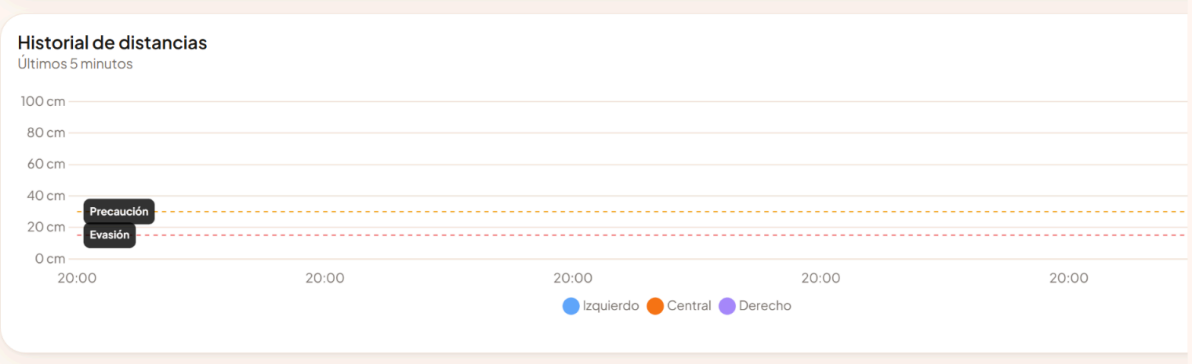


Figura 5: Historial de distancias: serie de líneas de los últimos 5 minutos, con las líneas de umbral `DIST_EVASION_CM = 15` y `DIST_PRECAUCION_CM = 30`. Sin datos porque el robot está fuera de línea en el momento de la captura.



Figura 6: Gauge de batería del robot activo.

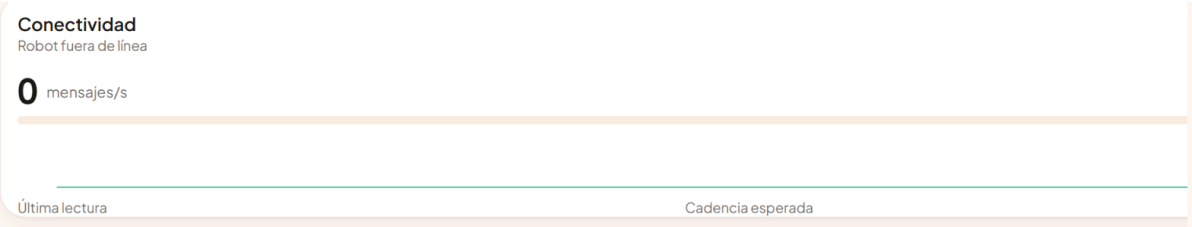


Figura 7: Tarjeta de conectividad: mensajes por segundo, última lectura y cadencia esperada.

7.3 Actualización en tiempo real

El dashboard se conecta al **namespace** `/live` de Socket.IO al cargar un robot: envía su JWT en el `handshake.auth.token`, se une a la sala `robot:{id}` y recibe de golpe el **estado inicial** (conexión, configuración y hasta 50 últimas lecturas en memoria), para no arrancar con el panel vacío esperando la próxima lectura. A partir de ahí, cada evento `telemetria` que emite el servidor actualiza las gráficas sin recargar la página ni volver a pedir datos por REST. Las animaciones de Chart.js están deshabilitadas globalmente (`Chart.defaults.animation = false`) porque compiten con actualizaciones a 2 Hz: una animación de 400 ms no llega a terminar antes de que entre el siguiente punto.

El reloj del navegador se sincroniza contra el del servidor (`sincronizar_reloj`) para que la latencia mostrada en la tarjeta de conectividad mida el retraso real del mensaje y no el desfase entre dos relojes de sistema distintos.

7.4 Interactividad

- **Zoom y desplazamiento** en las gráficas de series temporales (`chartjs-plugin-zoom`).
- **Filtros de rango e agregación** en el historial (`raw` / `minuto` / `hora`, fecha «desde»/«hasta»), con exportación a CSV del mismo rango filtrado.
- **Edición de configuración** del robot (velocidad de crucero, ángulos de sensor) desde el panel, que se envía por `actualizar_config` por WebSocket y se retransmite al robot como mensaje retenido en el tópico `config`.
- **Alta de robots** (`DialogoNuevoRobot`): genera el `ROBOT_ID` y el token MQTT, y entrega al usuario un `config.py` listo para copiar al firmware. El token se guarda cifrado (AES-256-GCM con clave derivada del `JWT_SECRET`), así que su dueño puede volver a consultarlo en «Credenciales» sin regenerarlo.
- **Modo oscuro** y vistas responsivas (comprobadas en 375 px, 768 px y 1440 px de ancho, ver anexos).

7.5 Manejo de errores

El cliente de socket reintenta la conexión automáticamente (reconexión nativa de Socket.IO) y la tarjeta de conectividad refleja en vivo si el robot está `en_linea` según el último evento de **Last Will** o de estado recibido. Si el **handshake** no trae un JWT válido, el middleware de Socket.IO rechaza la conexión antes de que se una a ninguna sala (`next(new Error('unauthorized'))`), y el cliente HTTP (`lib/api.ts`) traduce cualquier respuesta `4xx`/`5xx` de la API en un mensaje de error visible al usuario (vía `toast.tsx`) en vez de fallar en silencio.

8 Conclusiones

8.1 Resultados obtenidos

El sistema cubre de punta a punta los seis requisitos de evaluación: un nodo IoT con tres sensores y dos motores controlados por PWM, una base de datos relacional con reglas de integridad resueltas en el propio motor, inserción y lectura por API REST (tanto vía MQTT como vía HTTP directo), un script MicroPython que lee los sensores y envía los datos con reconexión automática, un sistema de autenticación propio con bcrypt + JWT, y un dashboard en React que visualiza la telemetría en tiempo real con gráficas de Chart.js y permite consultar el historial de sesiones. El diseño prioriza una latencia baja en el camino en vivo (emitir por WebSocket antes de tocar la base de datos) y una base de datos que **solo guarda lo que el hardware puede medir de verdad**, sin inventar campos «por si acaso».

8.2 Problemas y soluciones

Problema	Solución adoptada
El robot no tiene encoders ni GPS: no se puede calcular su posición de forma fiable.	Se eliminó del esquema toda columna de pose (`pos_x_cm`, `pos_y_cm`, `orientacion_deg`) en vez de simularla; el estado de movimiento se deduce del PWM real de los motores.
Guardar cada lectura inmediatamente en PostgreSQL añadía latencia al camino en vivo.	Se invirtió el orden: emitir por WebSocket primero, y solo encolar la escritura en un buffer que se vacía por lotes cada segundo.
Un mensaje MQTT mal formado (firmware con un bug) podía propagar una excepción y tumbar el proceso de la API.	Todo el camino de entrada (`manejar()`, `procesarTelemetria`) captura y registra cualquier error sin relanzarlo; el robot que lo causó sigue publicando en el siguiente ciclo sin reiniciar la API.
Un `estado_movimiento` de texto mantenido a mano por el firmware podía desincronizarse de lo que hacían realmente los motores.	Se sustituyó por las dos columnas de PWM con signo, y el estado se deduce siempre con la misma función (`movimiento()` en `@iot/shared`), igual en el backend y en el frontend.
Comparar bcrypt en cada lote de `POST /api/ingesta` (varias veces por segundo por robot) resultaba costoso para la CPU.	Caché en memoria del hash del token por robot, con una expiración de 5 minutos para que una regeneración de credenciales siga surtiendo efecto sin reiniciar la API.

Tabla 7: Problemas de diseño encontrados durante el desarrollo y cómo se resolvieron.

8.3 Aprendizaje logrado

El proyecto deja tres lecciones de ingeniería aplicables más allá de este robot concreto: primero, que la **latencia percibida** de un sistema en tiempo real depende del orden en que se hacen las cosas, no solo de la velocidad de cada pieza por separado (emitir antes de persistir). Segundo, que un esquema de base de datos es más robusto cuando **se niega a guardar lo que no se puede medir** en vez de rellenar

el hueco con una estimación poco fiable —la pose por odometría sin encoders siendo el ejemplo central —. Y tercero, que separar con claridad las identidades de un sistema (JWT de usuario del dashboard, token bcrypt del robot vía MQTT/HTTP) simplifica la seguridad más que intentar unificar todo bajo un único mecanismo de autenticación.

9 Anexos

9.1 Código fuente

Repositorio completo (monorepo `pnpm`): github.com/alvaro-tc/iot_dashboard.

Despliegue en producción, en vivo: iot-dashboard.tumype.com — cuentas de demostración en `docs/cuentas-de-prueba.md` (`alvaro@demo.com` / `cliente123` y `admin@demo.com` / `admin123`).

Secciones citadas a lo largo de este informe —

- Esquema de base de datos: `apps/api/db/schema.sql`, datos de demostración: `apps/api/db/seed.sql`.
- API REST y puente MQTT: `apps/api/src/routes/*.ts`, `apps/api/src/mqtt/bridge.ts`, `apps/api/src/ws.ts`, `apps/api/src/telemetria.ts`, `apps/api/src/auth.ts`.
- Firmware MicroPython: `firmware/main.py`, `sensores.py`, `motores.py`, `comunicacion_mqtt.py`, `conexion_wifi.py`, `radio_espnow.py`, `configuracion.py`.
- Dashboard: `apps/web/src/pages/`, `apps/web/src/components/`, `apps/web/src/lib/`.
- Geometría y simulación compartidas: `packages/shared/src/robot.ts`, `packages/shared/src/simulacion.ts`.
- Infraestructura de despliegue (VPS sin Docker): `infra/`.

9.2 Capturas de pantalla

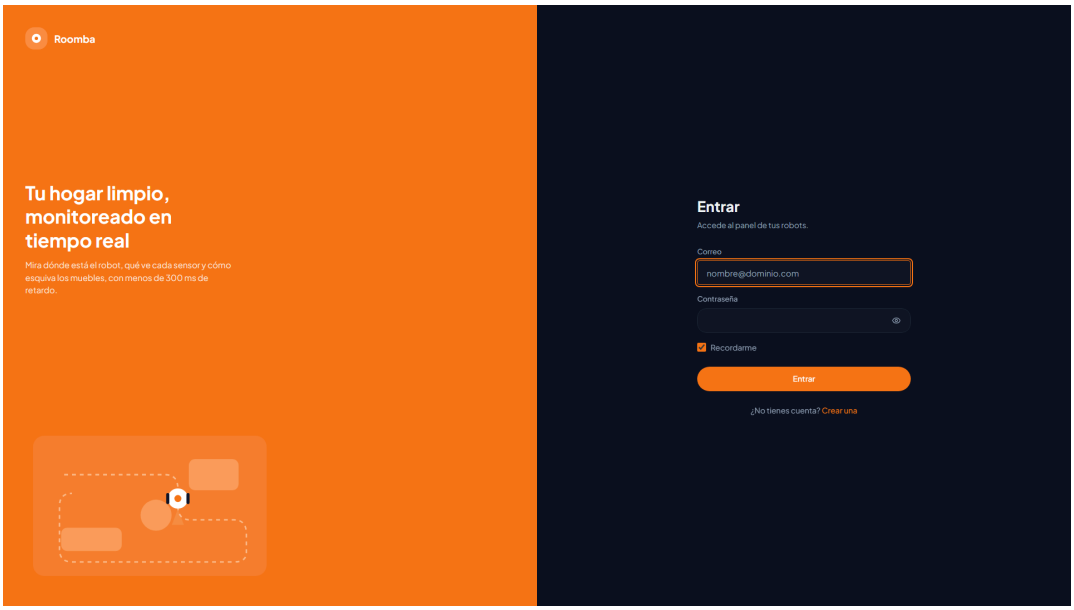


Figura 8: Pantalla de inicio de sesión.

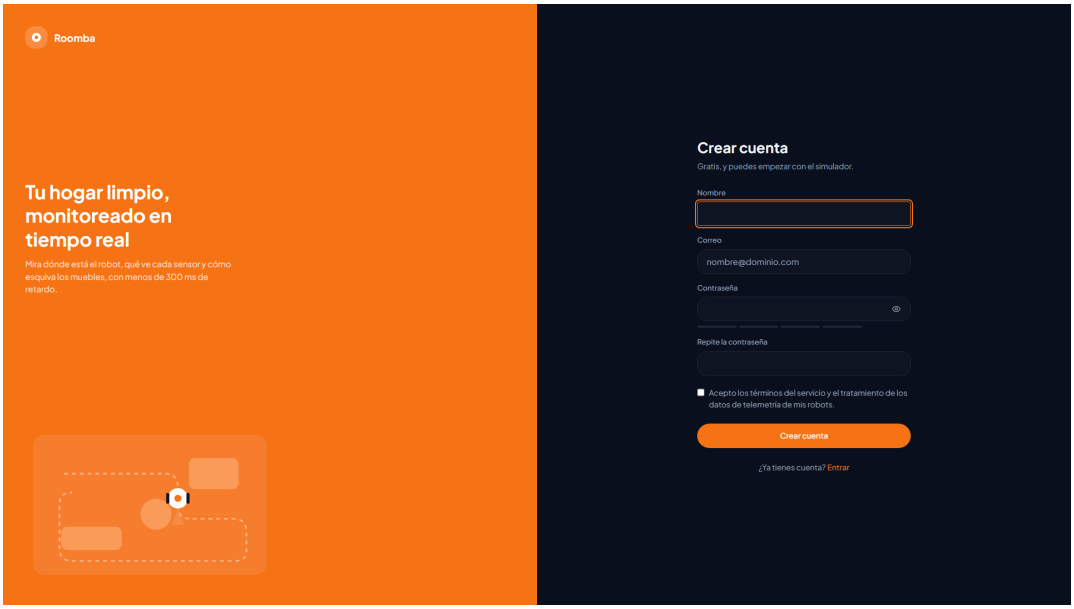


Figura 9: Pantalla de registro de una cuenta nueva.

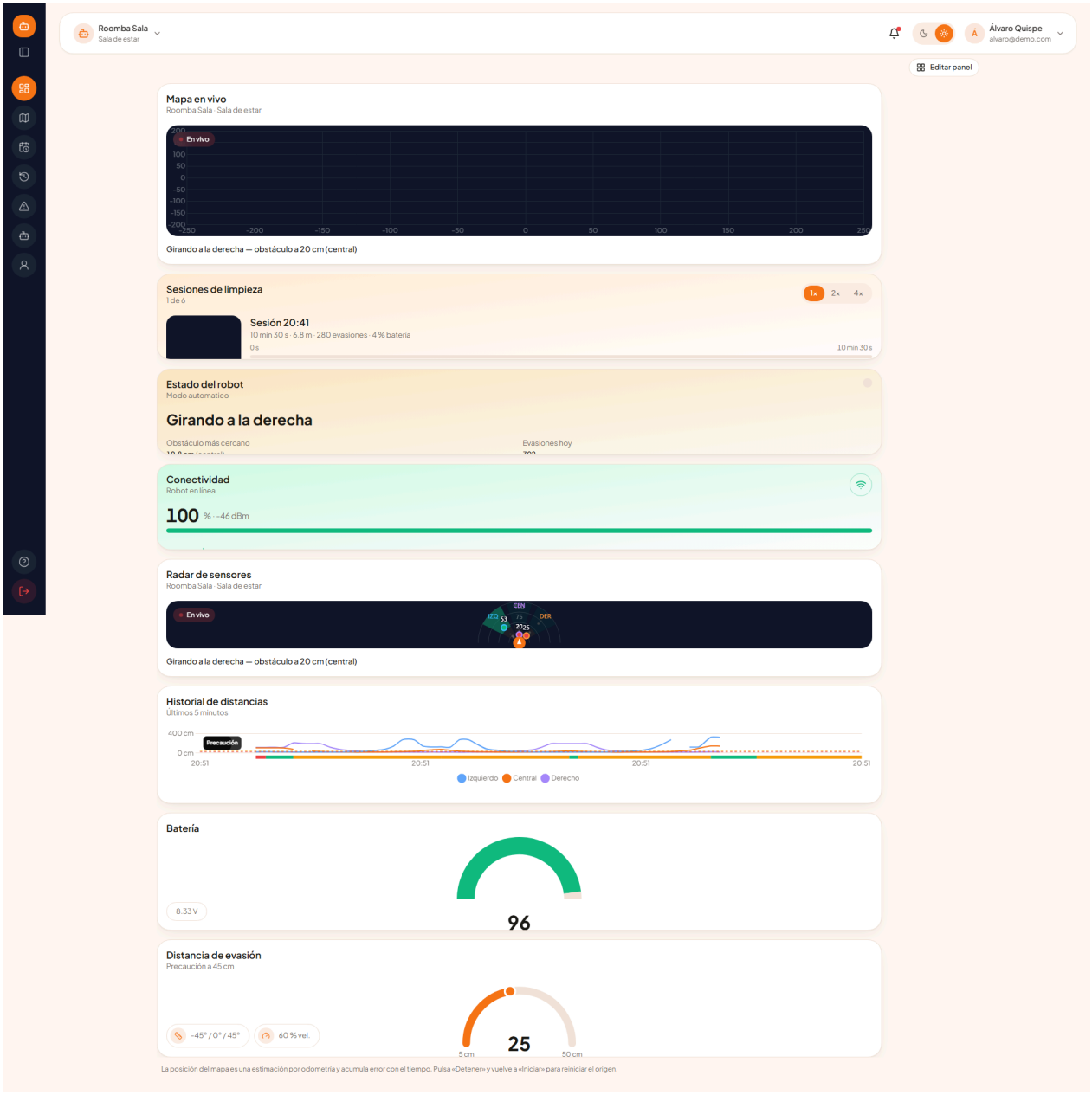


Figura 10: Panel en tiempo real de un robot: radar de sensores, series de distancia, batería, actividad de motores y conectividad.

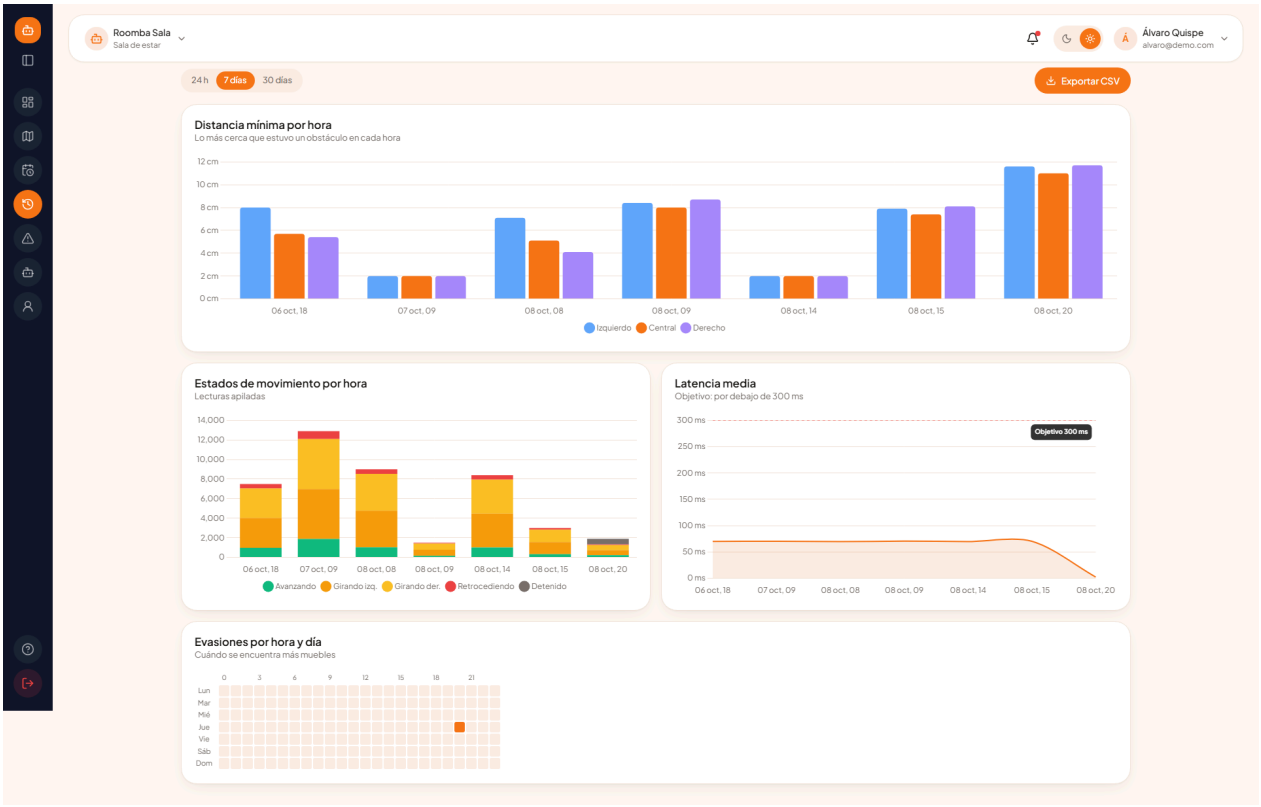


Figura 11: Historial de telemetría con filtros de rango y agregación.

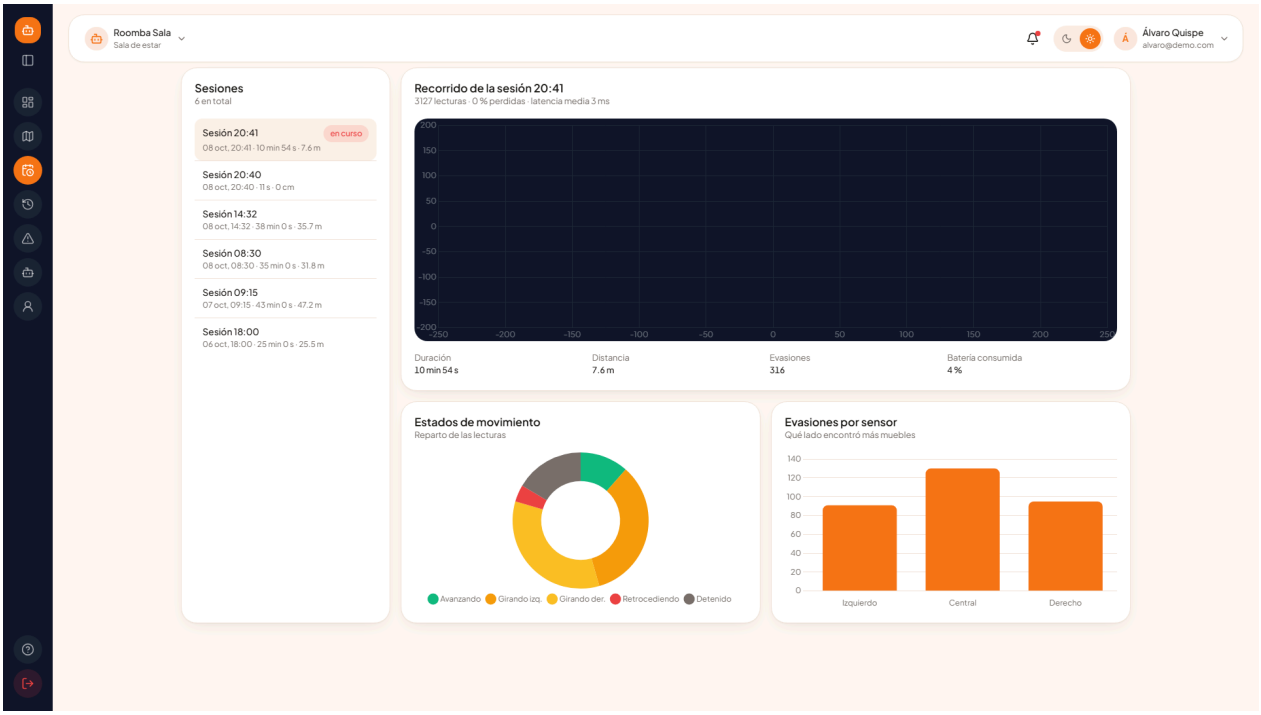


Figura 12: Listado de sesiones de limpieza con su resumen (duración, batería consumida, obstáculos cercanos por sensor).

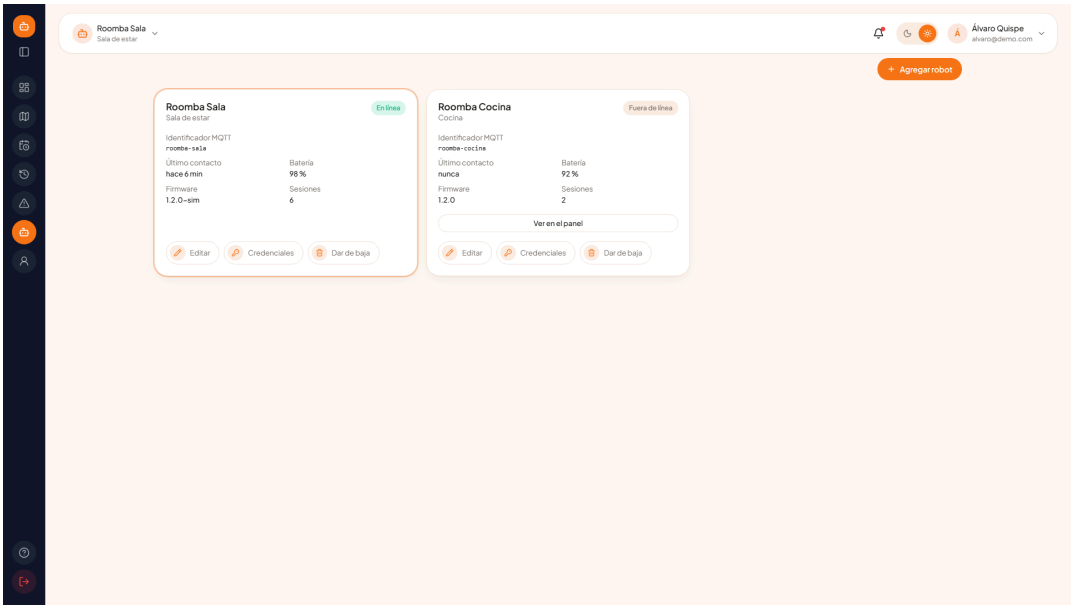


Figura 13: Gestión de robots del usuario: alta, token MQTT y estado de conexión.

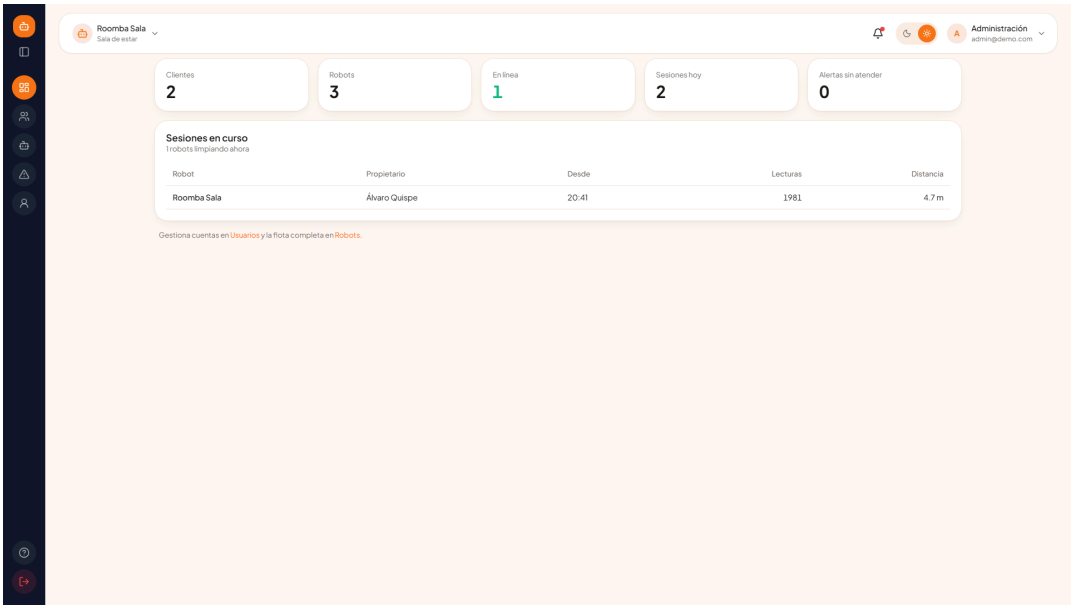


Figura 14: Panel de administración: usuarios y robots de todas las cuentas.

9.3 Capturas del despliegue en producción

Las siguientes capturas se tomaron directamente del despliegue en vivo (`https://iot-dashboard.tumype.com/`), entrando con las cuentas de demostración de `docs/cuentas-de-prueba.md` (`alvaro@demo.com` y `admin@demo.com`). Como en el VPS no hay un robot físico ni el simulador publicando en ese momento, los robots aparecen **fuera de línea** con batería y telemetría de la última sesión registrada — el estado real que vería cualquier visitante del enlace, sin maquillar.

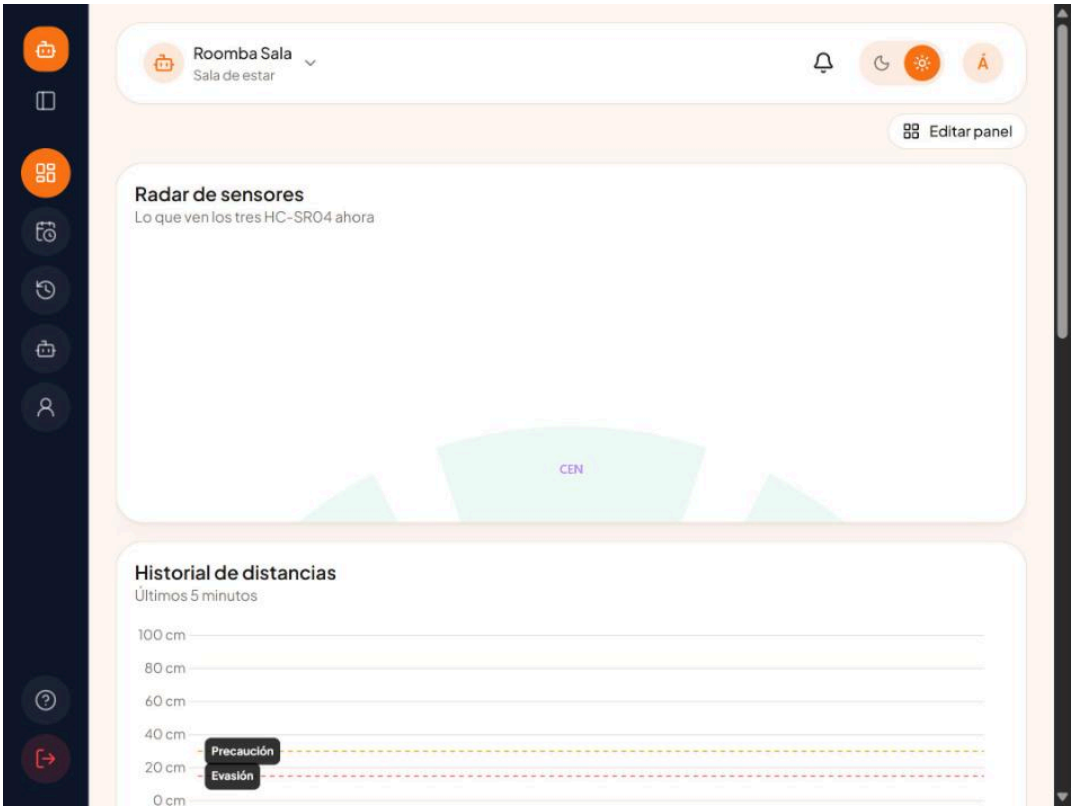


Figura 15: Panel en producción, cuenta `alvaro@demo.com`: radar de los tres sensores e historial de distancias de `Roomba Sala`.

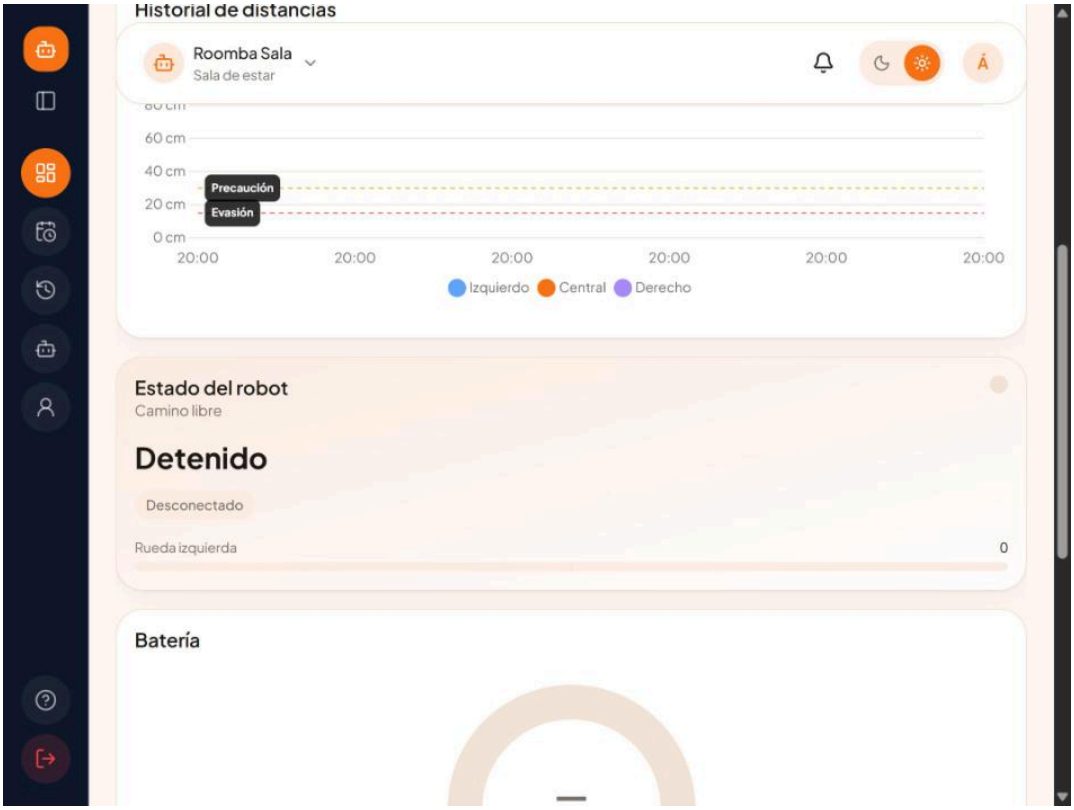


Figura 16: Panel en producción (continuación al desplazar): estado del robot, PWM de cada rueda y gauge de batería.

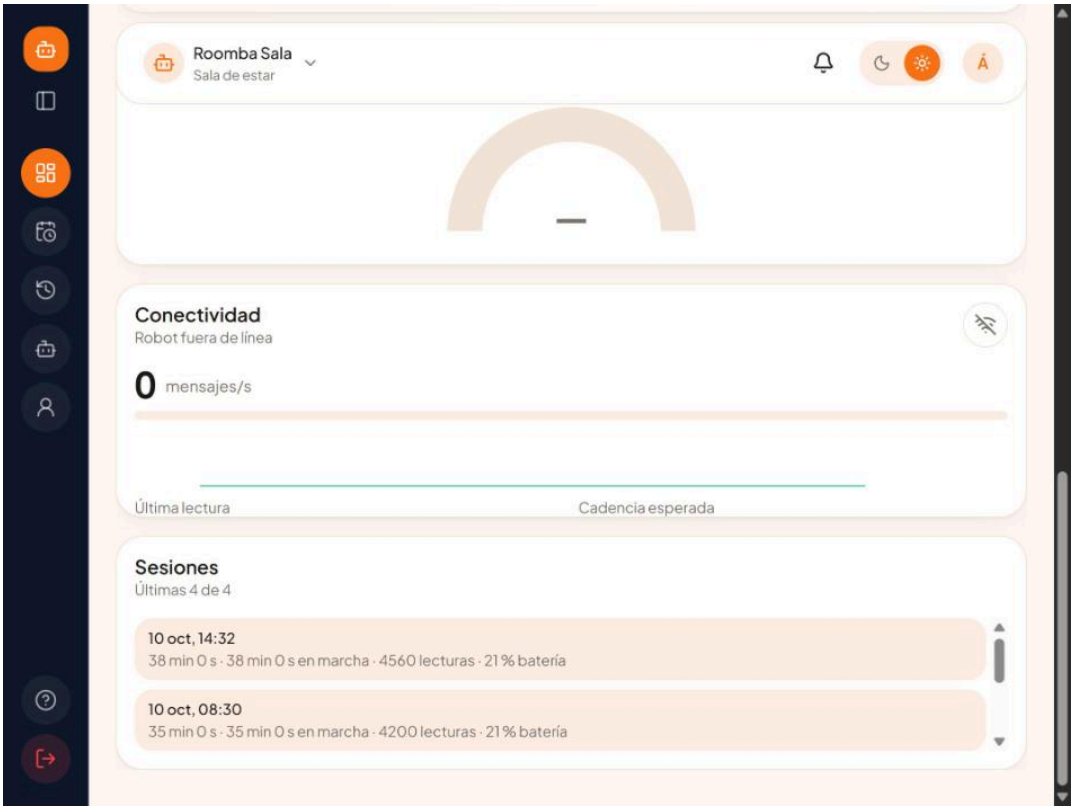


Figura 17: Panel en producción (continuación): tarjeta de conectividad (mensajes/s, cadencia esperada) y listado de sesiones recientes.

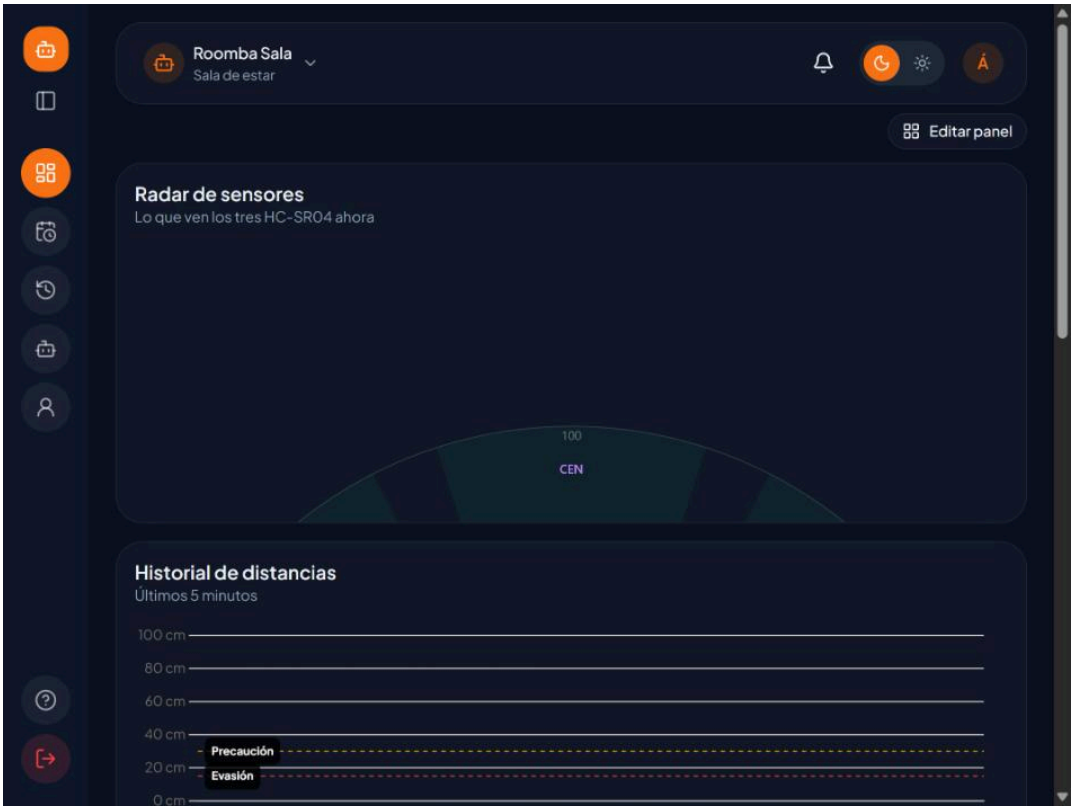


Figura 18: El mismo panel en modo oscuro.

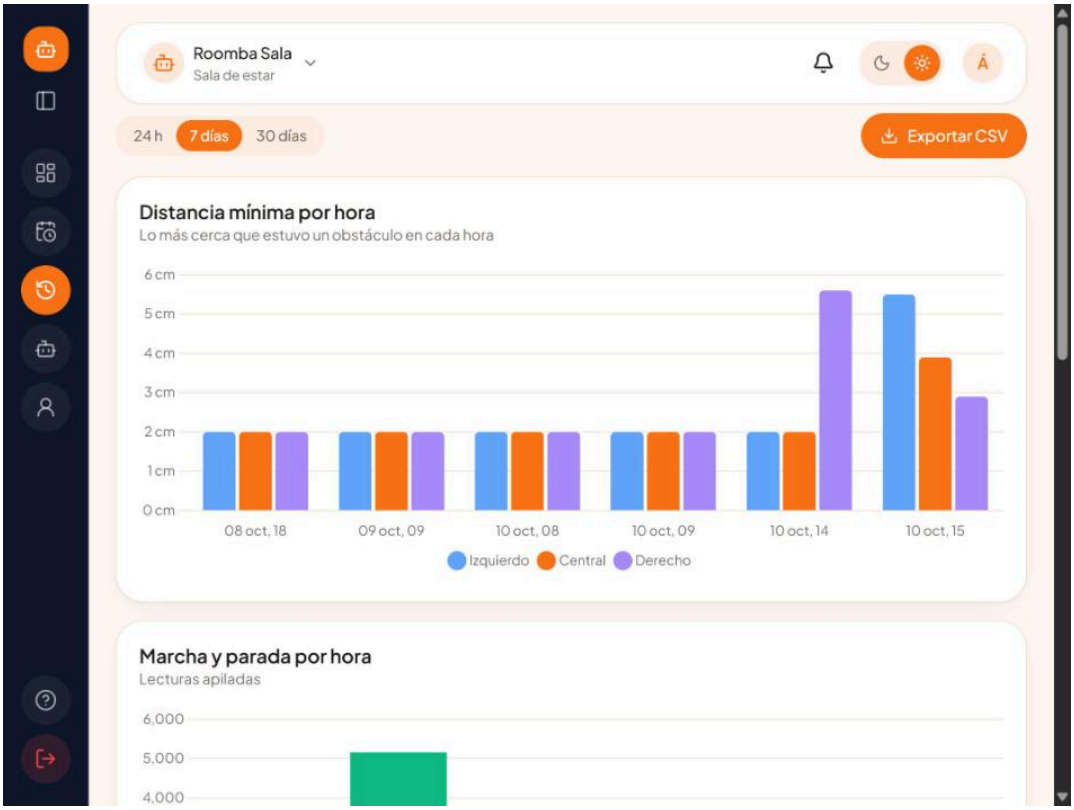


Figura 19: Historial en producción: distancia mínima por hora y marcha/parada, agregados en PostgreSQL por `v_lecturas_por_hora`.

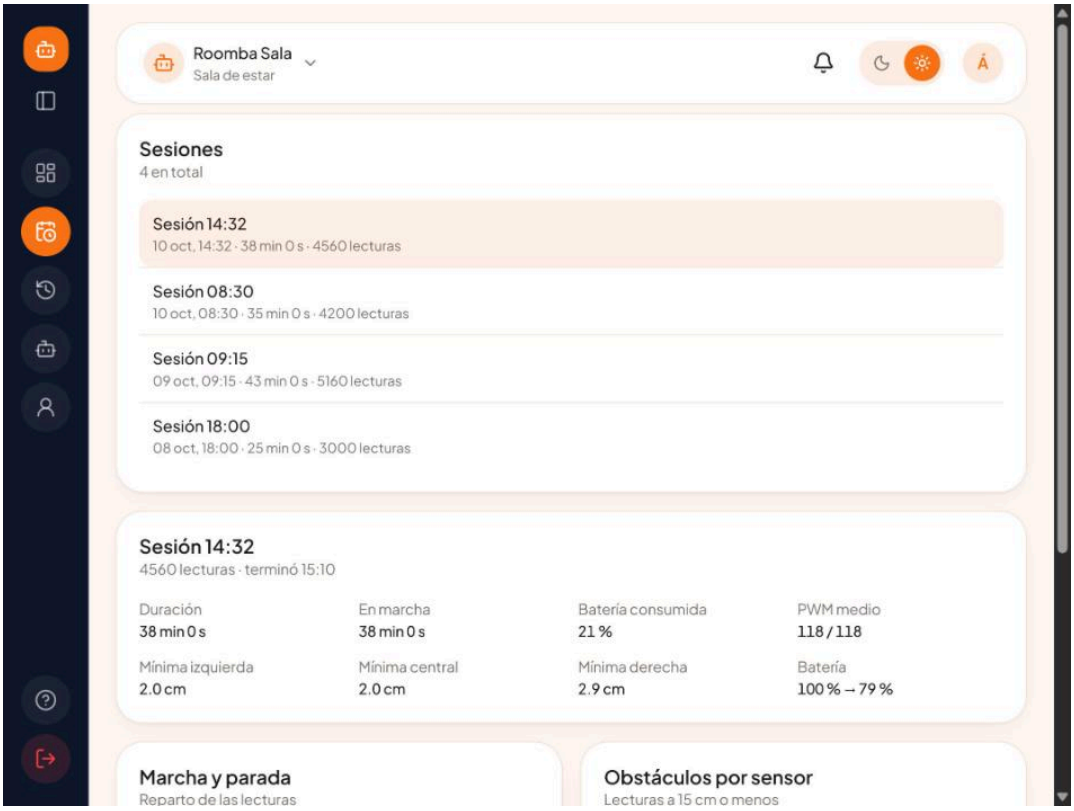


Figura 20: Sesiones de limpieza reales del robot de demostración, con su resumen de duración, batería consumida y PWM medio.

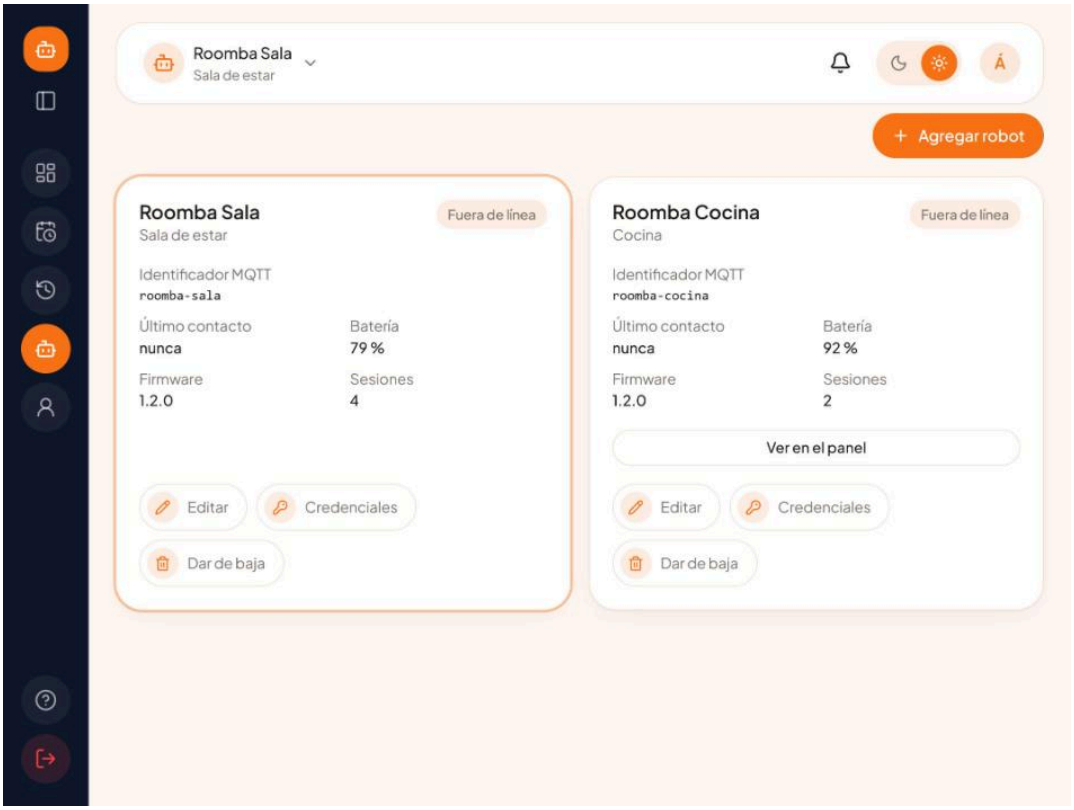


Figura 21: Robots de la cuenta `alvaro@demo.com` en producción: `roomba-sala` y `roomba-cocina`, ambos fuera de línea por falta de hardware/simulador conectado.

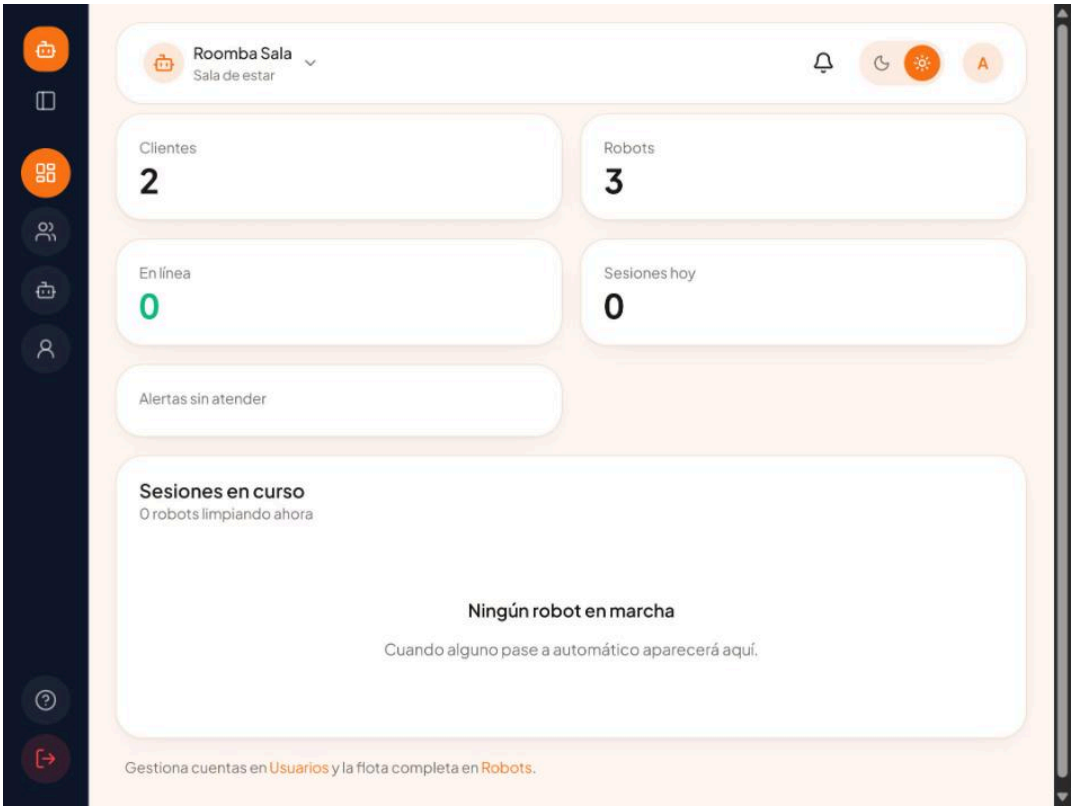


Figura 22: Panel de administración en producción (`admin@demo.com`): 2 clientes, 3 robots, 0 en línea y 0 sesiones activas en el momento de la captura.

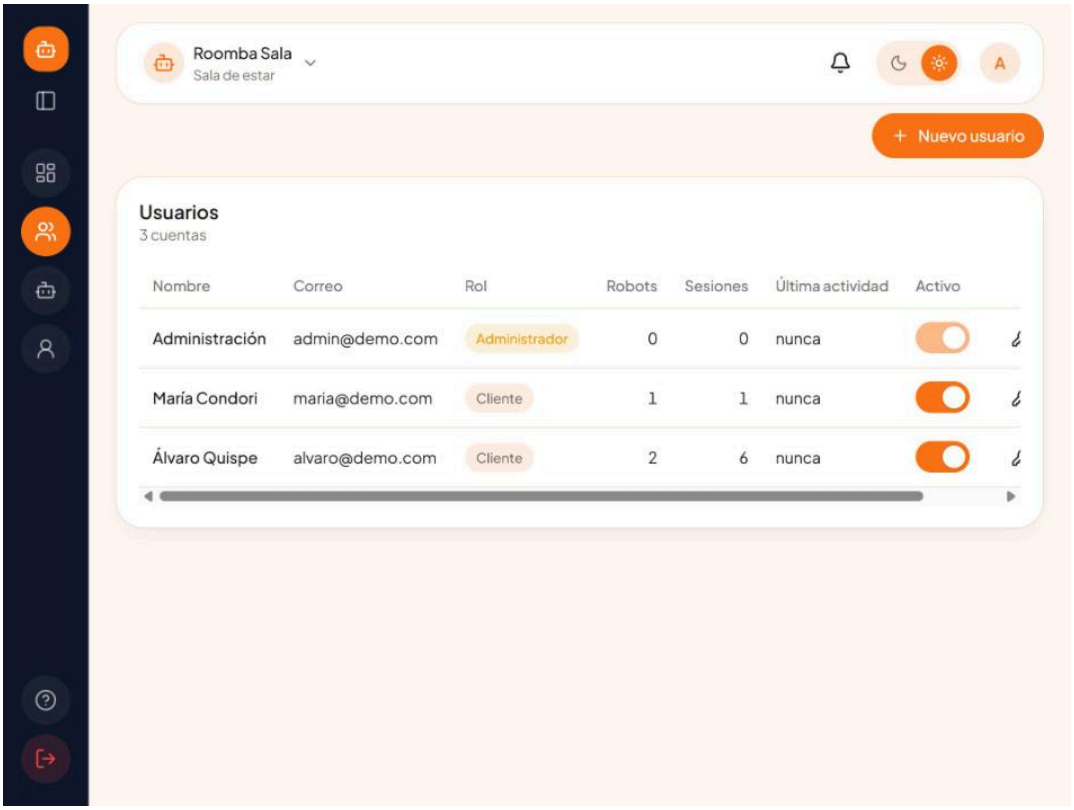


Figura 23: Gestión de usuarios en producción: las tres cuentas de demostración con su rol, número de robots y sesiones.

10 Referencias

- Repositorio del proyecto. https://github.com/alvaro-tc/iot_dashboard
- Despliegue en producción. https://iot-dashboard.tumype.com/
- PostgreSQL 16 — Documentación oficial. https://www.postgresql.org/docs/
- MQTT Version 5.0 — OASIS Standard. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- Eclipse Mosquitto — Documentación. https://mosquitto.org/documentation/
- Express.js — Guía de referencia. https://expressjs.com/
- Socket.IO — Documentación. https://socket.io/docs/v4/
- React — Documentación oficial. https://react.dev/
- Chart.js — Documentación. https://www.chartjs.org/docs/latest/
- MicroPython — Documentación para ESP32. https://docs.micropython.org/en/latest/esp32/quickref.html
- Espressif — ESP-NOW Protocol Guide. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
- JSON Web Tokens (RFC 7519). https://datatracker.ietf.org/doc/html/rfc7519
- bcrypt — **A Future-Adaptable Password Scheme**, Provos & Mazières (USENIX 1999).
- Zod — Validación de esquemas TypeScript. https://zod.dev/
- HC-SR04 Ultrasonic Ranging Module — hoja de datos del fabricante.
- TB6612FNG — hoja de datos, Toshiba (driver de puente H dual).